The MABEL Simulation Control Center: a Sim/Real-Unified Robot Studio on macOS

MABEL project

November 2, 2026

Abstract

The Simulation Control Center is the simulation twin of MABEL's on-robot control center: one browser page that combines a 3D robot viewer, full-body teleoperation, telemetry, a live map plane, autonomous navigation, and an episode recorder — with every signal on the page produced by a MuJoCo simulation served from a Mac, no ROS required. This document records the architecture that makes the sim and the real robot interchangeable behind one studio: a single-owner physics engine (`simulation_bridge`) that mirrors the hardware bridge's device contract, a shared-memory state channel (`[t, qpos, qvel, ctrl]` memmap) plus a JSON-RPC command channel with latched, exactly-once actuator-target delivery, and a pluggable *plant* seam through which the identical teleop controller drives either MuJoCo (`sim:bridge`) or the physical robot (`real:ros`). We quantify the platform: unthrottled physics steps cost 0.81 ms (sensors off) and 3.9 ms (with the 360-ray LiDAR ring) on an M-series Mac, so the paced 500 Hz loop ($\Delta t = 2$ ms) runs at 1.0× real time with $\approx$ 2.5× headroom; the mirror plane publishes a ∼23k-point colored depth cloud at 2 Hz; a full scene switch (re-launching the supervised sim stack) completes in 6–8 s. A library of **28 categorized scenes** — homes, restaurants, a supermarket, a gym, an airport, and a suite of *dynamic* city streets — is served through the same switcher; fourteen of them animate scripted people, vehicles, and conveyor belts through a kinematic actor runtime that costs 1.09 µs per actor and keeps 4.5× real-time headroom even on the busiest block. The macOS-native navigation and SLAM fallback that fills in for ROS 2 Nav2/`slam_toolbox` is specified and experimentally validated in the companion document `docs/macnav.pdf`.

Keywords: digital twin, MuJoCo, teleoperation, shared-memory IPC, sim-to-real, robot studio, occupancy mapping.

1 Overview

MABEL is a dual-arm mobile manipulator (3-module MAXSwerve base, cascaded lift, two 7-DOF arms with ORCA hands, actuated neck) that is teleoperated and trained from iOS, Vision Pro, and web clients. The project maintains two control centers: `control_center/` runs against the physical robot (ROS 2 on the Jetson), and `simulation_control_center/` — the subject of this document — runs the *same studio* against a simulated robot. The design goal is strict interchangeability: a client (or an experiment) cannot tell which robot it is talking to except by an explicit `kind: "sim"` tag.

The stack is organized as one physics owner plus independent read-side "planes":

- `simulation_bridge/engine.py` **(SimWorld)** — the ONE process allowed to call `mj_step` for a robot instance; the simulation twin of `hardware_bridge`'s device layer (§2).

- **Gateway** (`server/sim_teleop_bridge.py`) — hosts the teleop controller and the driver-arbitration/e-stop/deadman logic; talks to the world through the *plant* seam (§3); WebSocket `/teleop` + `/monitor` on port 9090 with `robot_state` at 50 Hz.
- **Camera server** (`server/sim_camera_server.py`) — renders the robot's cameras from the state memmap (MJPEG on 8080, UDP-JPEG on 9091); it never steps physics.
- **This backend** (`simulation_control_center/server`) — FastAPI app on port 8097 serving the studio page, the telemetry plane, the map plane, a browser MuJoCo viewer, the scene library, the native navigation service, and the episode save flow (§4).

2 One physics owner: `simulation_bridge` as the HAL mirror

2.1 The hardware-bridge contract, mirrored

On the real robot, `hardware_bridge` owns the devices: CAN to the arms, serial to the hands, UDP to the base. Everything else is a client of the state it publishes. `simulation_bridge` reproduces this ownership structure around MuJoCo: `SimWorld` accepts the same command surface — `set_twist` (body twist → swerve IK, the same `controller/mabel/swerve.py` used everywhere), `set_targets` (position setpoints for non-base actuators), `set_effort` (joint torques), `set_wrench` (Cartesian pushes), `set_estop`, `set_group_torque`, `reset` — with hardware semantics: a stale twist (older than 0.5 s) zeroes the base like a deadman, servo targets latch like a motor holding its setpoint, torque-disabled groups track the live joint so the limb yields, and e-stop zeroes drive while targets hold.

2.2 State out: the `[t, qpos, qvel, ctrl]` memmap

Bulk state is published per physics substep into a float64 memmap of length $1 + n_q + n_v + n_u$ (`/tmp/mabel_teleop_state.f64` for the default instance; `--instance` scopes the path so several robots can coexist on one machine). Legacy readers (camera server, desktop viewer, the control center mirror) map only the first $1+n_q$ floats and work untouched; newer consumers get true velocities and the executed actuator targets. A `.scene` sidecar names the loaded scene XML so read-side processes can re-create the identical model; a `.pert` memmap ($n_{\text{body}} \times 6$) is the desktop viewer's inbound Ctrl-drag wrench channel. This is single-writer/multi-reader shared memory: readers never block physics and a crashed reader cannot corrupt the world.

2.3 Commands in: JSON-RPC with exactly-once target delivery

Commands ride newline-delimited JSON over localhost TCP (port 8788; `simulation_bridge/rpc.py`): `{"id":n,"op":"set_targets","a":{...}}` → `{"id":n,"ok":true,"r":...}`. The engine's command store is latest-value with a *dirty set*: `set_targets` records the target and marks the actuator dirty; the physics thread pops the dirty set once per substep batch and writes `ctrl` only for changed actuators. Each commanded value is therefore applied *exactly once* (then held by the position servo), which matters for the torque-off tracking mode: a stale re-write of an old target would fight the "limb yields" behavior. Twists, efforts, and wrenches are deliberately *not* exactly-once — they are freshness-gated streams with per-channel timeouts (0.5/0.15/0.25 s), mirroring how the hardware treats streamed commands.

2.4 Single-owner locking and takeover

Constructing a `SimWorld` takes a per-instance `flock` (file `mabel_simulation_bridge_<inst>.lock` in `/tmp`) non-blockingly and refuses to start if another owner is alive — a stray script instantiating a world can never clobber the live robot's memmap (an accident that motivated the guard). The service wrapper (`simulation_bridge.py`) adds *takeover*: it signals the previous owner, waits for the flock to free, then starts; `--no-takeover` refuses instead. Side experiments run under different instance names — the experiment harness in the companion macnav document uses exactly this isolation.

3 The plant seam: one controller, two worlds

The gateway hosts the whole-body teleop controller in-process in both deployments; only the *plant* differs (`server/plant.py`). Each control substep runs

`plant.read_state()` → `cmd = teleop.update(pose, joy, dt)` → `plant.advance(cmd, ...)`

with three interchangeable plants: `MuJoCoPlant` (in-process `mj_step`, the original single-process sim), `SimBridgePlant` (commands to the simulation bridge over RPC, state from the memmap — the default sim path, `MABEL_DISPATCH_TARGET=sim:bridge`), and `ROSPlant` (state mirrored from encoders/odometry with `mj_forward` only — kinematics, no physics — and commands published on the canonical ROS contract; `real:ros`). The one-physics rule holds everywhere: exactly one `mj_step` owner exists per robot, and on the real robot nothing steps physics at all. Clients, driver arbitration, e-stop, episode recording, and the motion model are byte-identical across the seam.

4 The studio planes

4.1 Telemetry (`SimSource`)

The backend's telemetry plane is a passive WebSocket client of the gateway's own `robot_state` stream — every joint number shown is the actual MuJoCo state, not a parallel estimate. Velocities are finite-differenced; efforts and temperatures stay honestly blank because the sim publishes neither. Served to the page at `/ws/telemetry` at 15 Hz (59 DOF positions/velocities + targets + the LiDAR scan).

4.2 Map plane (`SimMirror`)

A background thread maps the bridge's qpos memmap, re-poses the same scene model with `mj_forward` (kinematics only — the one-physics rule again), and derives: (i) the 360-ray planar LiDAR scan from the model's rangefinder ring; (ii) chassis odometry; (iii) a colored head-depth point cloud (∼23k points at 2 Hz), rendered *demand-driven* — macOS serializes GL across processes, so the renderers exist only while a `/ws/map` client is actually watching. Wire records are the exact `server/map_stream.py` binary protocol (occupancy 0x04, path 0x03, odometry, cloud), so the page code is shared with the ROS-fed real-robot map plane. The scan also feeds the native navigation service (`server/macnav`; see `docs/macnav.pdf`) whose SLAM grid and A* path are published back onto this same plane.

4.3 Browser MuJoCo viewer

`/ws/simview` streams a server-rendered interactive MuJoCo view into the page, so the studio needs no local MuJoCo install on the client; the desktop viewer (`--viewer`) remains available and its Ctrl-drag perturbations enter the world through the `.pert` channel.

4.4 Scene library and safe respawn

The library holds **28 scenes** registered in `simulation/mabel_mujoco/scenes/scenes.json`, each with a robot-free thumbnail and a `category`, grouped in the picker into *Home & Living* (home, two more living rooms, bedroom, washroom, the one-floor full house), *Food & Dining* (Franka kitchen, two cafes, casual restaurant, fast food, fine dining), *Work & Public* (office, supermarket, gym, airport), *Industrial* (factory, assembly line, warehouse), *Outdoor & Leisure* (park, golf, front yard, rooftop lounge), and *City & Streets* (city sidewalk, signalized intersection, grand boulevard, city plaza). The nine original scenes are hand-authored MJCF; the nineteen added on top are emitted by a small builder package (`simulation/mabel_mujoco/scripts/tools/scenegen`) from shared prefabs (rooms, furniture, storefronts, vehicles, people) so the set stays consistent and regenerable via `gen_extra_scenes.py`. An acceptance harness (`verify_scenes.py`) loads every scene, checks the spawn is penetration-free, settles it 200 steps for stability, and (§4.5) validates each dynamics sidecar; all 28 pass. Picking a scene restarts the supervised sim stack into that world; if the studio had attached to an externally-started sim it *adopts* it (the launcher's port pre-clean evicts the old bridge), and the page auto-refreshes when the new world is up, so switching is a single click regardless of how the sim was started. The sim stack is launched with a filtered relay config that tunnels *only* the sim studio (`mabel_sim_studio`, :8097 → `control-sim`); it never registers the `mabel_control`/`mabel_video` rathole services that belong to the real robot's Control Center (`control`), so a scene switch can never seize the physical robot's tunnel — the two studios stay strictly separate on the relay.

"Reset Elsewhere" (`server/scenes.py::safe_spawn`) rasterizes every environment geom's XY footprint over the robot's standing z-band onto a 10 cm grid, inflates by a 0.8 m clearance radius, picks a random far free cell (seeded RNG in tests), then *verifies in MuJoCo* by placing the base free joint and rejecting any robot–environment penetration before the bridge teleports the robot. The same footprint logic supplies the ground-truth occupancy rasters for the navigation experiments in the companion document.

4.5 Scripted scene dynamics

Fourteen scenes ship a `mabel_<scene>.dynamics.json` sidecar that brings the world to life: pedestrians walk the sidewalks and concourses, cars and a bus drive their lanes, the assembly-line conveyor carries its parts. The runtime (`simulation_bridge/dynamics.py`) is a client of the one physics owner, not a second one: actors are free-jointed, gravity-compensated bodies whose pose the engine writes *kinematically* each substep, so the memmap, cameras, web viewer and rig mirror all see the motion with no new plumbing, and a collision shoves the robot rather than derailing the actor (an actor has effectively infinite inertia). Only the torso/chassis geom collides (masked to the robot, `contype` = 2); swinging limbs and spinning wheels are visual-only, keeping the contact broadphase lean.

Motion is planned to look natural and is fully deterministic (no RNG): each actor follows a corner-filleted polyline at an acceleration-limited speed that is capped by $\sqrt{2a\,(d-h)}$ toward (i) red stop-lines on period/offset/dur cycles — traffic lights, bus dwells, window-shopping pauses — and (ii) the *robot* when it stands in the actor's path corridor, so cars brake for MABEL and pedestrians wait (then, past a patience timeout, squeeze past). Leg/arm gait and wheel spin are phase-coupled

to v, so an actor glides to a halt mid-stride instead of moonwalking. To keep the physics loop cheap the controller recomputes actor targets at only 50 Hz (`KEY_DT`); the 500 Hz integrator carries the written velocity between keyframes.

Figure 1 quantifies the cost against every dynamic scene with the full robot loaded, in the bridge's deployed configuration (LiDAR ring off). The keyframe write is negligible — 1.09 µs per actor, so even the 12-actor city block adds 12 µs on top of a 0.44 ms substep — and the per-substep cost is set by scene geometry, not actor count, leaving 4.5× real-time headroom in the worst case. Figure 2 shows the resulting motion is believable: a car decelerates smoothly to a full stop at the red and accelerates back to cruise (no teleport), the walking gait amplitude tracks speed ($r = 0.97$), and the conveyor carries its part at 92 % of belt speed by friction alone. The harness that regenerates both figures is `simulation_control_center/experiments/exp_dynamics.py`.

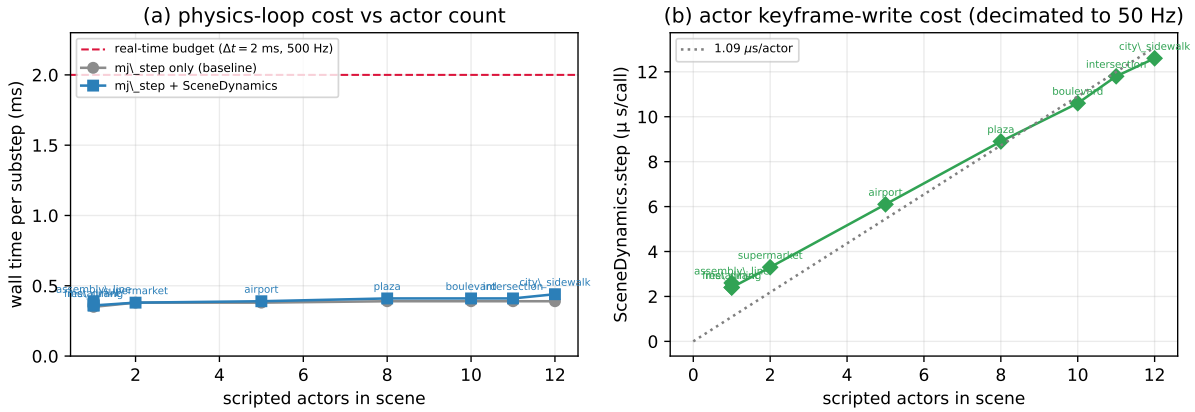


Figure 1: Scene-dynamics cost with the full robot loaded (sensors off, as deployed), across the dynamic scenes ordered by actor count. (a) `mj_step` alone (baseline, actors present but undriven) versus `mj_step` + `SceneDynamics.step`: the two are indistinguishable and both stay far under the 2 ms real-time budget — cost is driven by scene geometry, not actors. (b) The actor keyframe write in isolation: 1.09 µs per actor (50 Hz decimated), i.e. tens of microseconds even for the busiest city block.

4.6 Episode save flow

Record → Stop stages an episode (7 RGB cameras, stereo depth, qpos, qvel, ctrl, LiDAR, odometry, teleop commands, all sampled in *sim time* so GPU render stalls never skew the data); a confirm dialog shows length, frame count, rate, modalities, and on-disk size, and lets the operator rename, re-folder, or discard. Default names are environment-stamped and sequential (`home_20260702_003`). The recorder consumes a source interface (`server/sim_sensor_source.py`) that a hardware source can replace one-for-one.

4.7 Teleop surface and webcam body teleop

The 3D viewer's operator surface follows the iOS-v2 grammar: one `Start Teleop` verb opens the session and reveals a three-way control space (`Drive · Joint Space · Task Space`); the stiff/soft behaviour toggle is always visible; camera-view controls (3rd/1st POV above recenter/fit/follow) sit bottom-center; and the navigation cluster (top-left) arms a pose-target cursor with which goals are dropped and aimed on the map. The studio can also mimic the Vision Pro input path using the

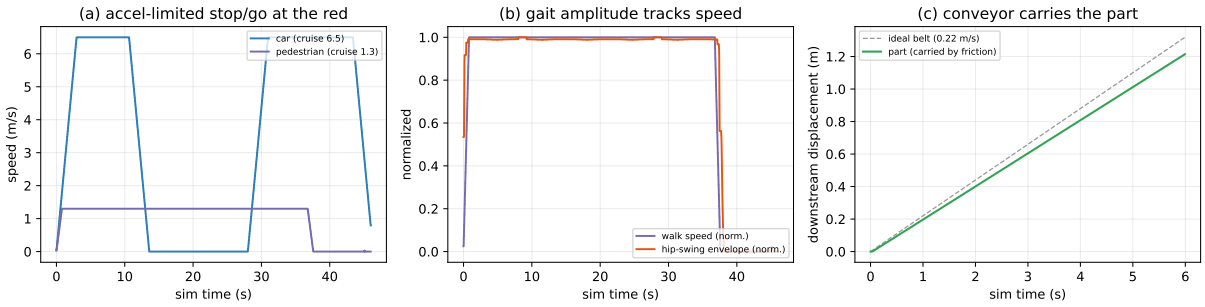


Figure 2: Motion naturalness (deterministic, at the signalized intersection). (a) A car and a pedestrian brake to a stop at the red on an acceleration-limited envelope and accelerate back to cruise — not a teleport. (b) The walker's hip-swing amplitude envelope tracks its speed ($r = 0.97$), so it slows mid-stride. (c) The assembly-line conveyor carries a part downstream at 92 % of belt speed through contact friction, with the drive on a slide joint wrapped by one texture period so the belt looks endless.

desktop webcam (MediaPipe pose + hands), engaged as the live session's `Body` input source: arms, fingers, and head-gaze always; torso lean and whole-body control as more of the operator becomes visible. This produces the same `teleop_frame` wire messages as the headsets, so the arbitration and motion model treat a webcam operator exactly like any other driver.

4.8 Choreography timeline (long-horizon policy composer)

The bottom band of the page is an NLE-style timeline with two lanes (NAVIGATION / MANIPULATION) on which typed *action blocks* — a live-validated whole-robot motion library (emotes, dances, scripted task policies), plant-calibrated parametric nav primitives, a REST-backed closed-loop move-to-goal, and static poses that double as keyframes — are bladed, trimmed, snapped, duplicated, and executed as one 20 Hz choreography over the same `/teleop` wire as any client. A location-aware validator tracks the robot's implied position along the timeline and auto-inserts move-to-goal blocks where an action is scheduled at the wrong place. The composer's data model, the easing mathematics and velocity budget of the motion library, and its full live-simulator validation campaign (nav calibration, tracking, base pinning, end-to-end choreography) are specified in the companion document `docs/choreography.pdf`.

5 Measured platform numbers

All numbers measured on the development M-series Mac (Apple silicon, single-process, `mabel_home` scene, $n_q = 116$, $n_v = 108$, $n_u = 59$, $\Delta t = 2$ ms), 2026-07; measurement scripts are the probes and the harness of the companion macnav document (`simulation_control_center/experiments/exp_macnav.py`).

The 500 Hz loop self-paces on a real-time accumulator with a bounded catch-up budget (50 ms of physics per wake), so a scheduling hiccup slows sim time rather than exploding a physics step. A separately documented finding (repo memory, camera-contention experiments): the camera server can starve the control loop via CPU scheduling; the deployed mitigation is `taskpolicy -c utility` on the camera process.

Quantity	Value	Condition
`mj_step` + publish, sensors off	0.81 ms/step	200-step mean, unthrottled
`mj_step` + publish, LiDAR ring on	3.86 ms/step	360 rangefinder rays
On-demand scan (toggle + `mj_forward`)	3.9 ms	experiment-harness path
Paced physics loop	500 Hz = 1.0× RT	2.5× headroom, sensors off
Depth cloud	∼23k pts at 2 Hz	demand-driven, head camera
Scene switch (stack relaunch)	6–8 s	supervised restart + reattach
Robot settle after spawn	≈0.5 s	spawn 9 cm above contact
Scene-dynamics keyframe write	1.09 µs/actor	§4.5, 50 Hz decimated
Busiest dynamic scene (city sidewalk)	0.44 ms/step = 4.5× RT	12 actors, full robot, sensors off
Actor tax at 12 actors	12 µs/step	0.6% of the 2 ms budget
Conveyor carry efficiency	92 % of belt speed	friction transport

Table 1: Measured platform performance. The LiDAR-ring cost is why the engine compiles sensors in but disables them unless `MABEL_SIM_SENSORS=1`: nothing in the default stack consumes `sensordata` from the physics process (the mirror re-derives the scan read-side).

6 Implementation and usage

```
./simulation_control_center/run.sh              # sim (home scene) + studio -> :8097
./simulation_control_center/run.sh --viewer     # + native MuJoCo desktop window
./simulation_control_center/run.sh -s kitchen   # start in another scene (name or .xml)
./simulation_control_center/run.sh --gui-only   # attach to a sim you started yourself
./simulation_control_center/run.sh --mock       # GUI only, synthetic plant (CI/dev)
```

On Ubuntu with a built `mabel_ws`, `run.sh` launches the full ROS sim stack (cuVSLAM + nvblox + Nav2) instead and points the map plane at the ROS feed (`MABEL_MAP_PORT=9092`); on macOS the built-in mirror plane and the native macnav service take over. One Ctrl-C stops the studio and the sim it launched.

Port	Plane	Notes
8097	studio + backend	`/api`, `/ws/telemetry`, `/ws/map`, `/ws/simview`
9090	gateway WebSocket	`/teleop`, `/monitor`; `robot_state` at 50 Hz
8080	cameras (MJPEG)	head/front/wrist renders
9091	cameras (UDP-JPEG)	headsets
8788	simulation bridge RPC	ndjson over localhost TCP
—	state memmap	`/tmp/mabel_teleop_state.f64` (+`.scene`, `.pert`)

Table 2: Port map of the sim stack (defaults; all env-overridable).

A public relay (`sim.mabel.example.com`) tunnels the studio same-origin through the project VPS, falling back to an always-on mock twin when the Mac is offline.

7 Limitations

- **No ROS on macOS.** Nav2, `slam_toolbox`, cuVSLAM and nvblox cannot run; the native fallback (`server/macnav`) covers 2D LiDAR SLAM and navigation but not visual SLAM or 3D reconstruction, and has *no loop closure* (drift quantified in `docs/macnav.pdf`).

- **Effort channel is sim-only so far.** `set_effort` maps to `qfrc_applied`; the real robot's drivers do not yet expose a torque interface on this contract, so WBC compliance modes validated in sim are not yet portable.
- **Sim telemetry is honest but incomplete.** Efforts and temperatures are blank; the "sensor health" analogs are fixed ambient values.
- **macOS GL serialization.** Off-screen rendering (cameras, depth cloud, thumbnails) shares one GPU queue across processes; heavy rendering slows other renderers (not physics). The mirror's demand-driven renderers and the recorder's sim-time sampling are the mitigations.
- **AABB footprints.** `safe_spawn` (and the ground-truth rasters derived from the same logic) over-approximate rotated geoms by axis-aligned boxes; respawn poses are conservative, and map-quality metrics against these rasters inherit the bias (discussed in the companion document).

Companion documents

`simulation_control_center/docs/macnav.pdf` — methods and experimental validation of the native navigation/SLAM stack. `simulation_control_center/docs/choreography.pdf` — the timeline policy composer and the validated motion library. `simulation_control_center/docs/retargeting.pdf` — webcam body teleop. `controller/docs/mabel_controller.tex` — the teleop controller and motion model. `controller/docs/wbc_stability.tex` — tip-over stability. `simulation_bridge/README.md`, `simulation_control_center/README.md` — operational docs.

References

[1] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2012.

[2] S. Macenski, F. Martín, R. White, and J. Clavero, "The Marathon 2: A navigation system," in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2020.

[3] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, 6(61):2783, 2021.

[4] M. Quigley *et al.*, "ROS: an open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, 2009.

[5] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with Rao-Blackwellized particle filters," *IEEE Transactions on Robotics*, 23(1):34–46, 2007.